GENOMIC MUTATION TRACKER

Complete Project Guide

This guide covers everything built in the project — every file, every concept, every doubt answered. Read this before any interview.

1. Project Overview & Flow

The Genomic Mutation Tracker is a C++ system that searches patient DNA sequences for known disease-causing mutation patterns using two classic string algorithms, computes risk scores, and stores results in MySQL for analytics.

The Complete Flow

```
STEP 1: generator.cpp runs
  → reads config (100 patients, 1000 char DNA)
  → generates random DNA using rand()
  → injects mutations based on patient age
  → writes patients.txt

STEP 2: main.cpp runs
  → reads patients.txt into vector
  → KMP searches all 5 mutations in each patient
  → Rabin-Karp searches all 5 mutations
  → timing comparison printed
  → computeStats() → density, unique count, risk score
  → computeFrequencies() → global mutation counts
  → printTopK() → most common mutations
  → insertToMySQL() → writes 3 CSV files

STEP 3: insert_db.py runs
  → connects to MySQL
  → clears old data
  → inserts from results.csv → patients table
  → inserts from mutation_results.csv → mutation_results table
  → inserts from frequencies.csv → mutation_frequencies table

STEP 4: MySQL Workbench
  → run analytics queries from setup.sql
  → see top risk patients, age group analysis, rankings
```

Tech Stack

Language/Tool	Purpose	Why chosen
C++	Core algorithms + data processing	Fastest for string operations at scale
Python	MySQL database insertion	mysql-connector works without arch issues
MySQL	Permanent storage + analytics	SQL window functions for rankings
Git/GitHub	Version control	Industry standard

2. File Structure

Here is every file in the project and what it does:

File	Type	Purpose
generator.cpp	C++	Generates 100 fake patients with age-correlated DNA mutations, writes patients.txt
patient.h	C++ Header	Defines the Patient struct — shared by main.cpp, db.cpp, db.h
kmp.cpp	C++	Implements buildLPS() and KMPsearch() functions
kmp.h	C++ Header	Declares KMP functions so main.cpp can use them
rabin_karp.cpp	C++	Implements RabinKarpMulti() — searches all patterns grouped by length
rabin_karp.h	C++ Header	Declares RabinKarpMulti() function
main.cpp	C++	Orchestrates everything — loads patients, runs both algorithms, computes stats, writes CSV
db.cpp	C++	Writes results to 3 CSV files: results.csv, mutation_results.csv, frequencies.csv
db.h	C++ Header	Declares insertToMySQL() function
insert_db.py	Python	Reads 3 CSV files and inserts all data into MySQL database
setup.sql	SQL	Creates database and tables. Also contains all 5 analytics queries
patients.txt	Data	Generated input — 100 patients with ID, age, DNA string
results.csv	Data	Output — patient stats and risk scores
mutation_results.csv	Data	Output — every mutation found at every position
frequencies.csv	Data	Output — global count of each mutation across all patients
.gitignore	Config	Tells git to ignore .exe and .o files
README.md	Docs	Project description for GitHub viewers and recruiters

3. generator.cpp

Generates 100 simulated patients and writes them to patients.txt. Runs once before the main program.

Key Functions

generateDNA(length)

Creates a random DNA string of given length using only A, T, C, G characters.

```
string bases = "ATCG";
for each position:
  rand() % 4 → gives 0,1,2,3
  bases[0]='A', bases[1]='T', bases[2]='C', bases[3]='G'
  picks one character randomly
  appends to dna string
```

injectMutation(dna, mutation)

Overwrites a random section of DNA with a mutation pattern. Uses OVERWRITE not INSERT so DNA length stays constant.

```
maxPos = dna.length - mutation.length
  (so mutation fits without going out of bounds)
pos = rand() % maxPos
  (random start position)
for each char in mutation:
  dna[pos + i] = mutation[i]
  (overwrite character by character)
```

mutationsForAge(age)

Returns how many mutations to inject based on age. This creates a biologically motivated correlation.

```
age < 30  → rand() % 2      → 0 or 1 mutations
age < 50  → rand() % 3      → 0, 1 or 2 mutations
age < 70  → 1 + rand() % 3  → 1, 2 or 3 mutations
else      → 2 + rand() % 3  → 2, 3 or 4 mutations
```

SIMULATION NOTE: Age and DNA are not truly linked biologically. We impose this correlation to make SQL analytics meaningful. In production, real NCBI genomic data would reveal natural patterns.

Why overlaps are allowed

When injecting multiple mutations, a later injection can overwrite an earlier one. This is intentional — it creates natural variability. Same age can give different mutation counts, just like real patients. Age influences probability, not a guaranteed fixed count.

Why rand() needs srand(time(0))

```
srand(time(0))  → seed set ONCE at program start
                  time(0) = seconds since Jan 1 1970
                  changes every second
                  different seed = different sequence

Without srand():
  C++ uses default seed = 1
  same sequence every run
  same 100 patients every time → useless

rand() chain:
  each call uses PREVIOUS result as input
  seed only touches call #1
  rest of chain runs automatically
  Patient 2 starts from where Patient 1 ended
```

4. patient.h — The Patient Struct

A header file created to share the Patient struct across multiple .cpp files without duplication.

Why we moved it out of main.cpp

Originally Patient was defined inside main.cpp. When db.cpp was created, it also needed to know what a Patient is. Two options:

```
Option A: Copy struct into db.cpp too
  Problem: if you add a field, you must update in 2 places
  Leads to bugs — files get out of sync

Option B: Move to patient.h (what we did)
  main.cpp    includes patient.h
  db.cpp      includes patient.h
  Everyone reads from ONE place
  Change in one file = everyone gets update
```

The struct fields explained

Field	Type	What it stores
id	string	Patient ID like P001, P002
age	int	Age between 20 and 80
dna	string	DNA sequence (1000 chars of A,T,C,G)
totalMutations	int	Total mutation occurrences found by KMP
mutationMap	map<string,vector<int>>	GGT->[23,145], ATCGG->[726] etc
rkMutationMap	map<string,vector<int>>	Same but from Rabin-Karp
rkTotalMutations	int	Total count from Rabin-Karp
uniqueMutations	int	How many different mutation types found
mutationDensity	double	Mutations per 1000 characters
riskScore	double	Computed risk score

What is map>?

```
map links a KEY to a VALUE
string key   = mutation name  ("GGT")
vector value = all positions  ([23, 145, 302])

mutationMap["GGT"] = [23, 145, 302]
  means GGT was found at positions 23, 145, and 302

mutationMap["ATCGG"] = [726]
```

```
  means ATCGG was found at position 726 only

Why map and not unordered_map?
  map → keys always sorted alphabetically
  output always in same order (ATCGG before GGT before TAGG)
  easier to read and compare
  only 5 mutations → speed difference negligible
```

5. kmp.cpp / kmp.h — KMP Algorithm

Why KMP instead of naive search?

```
Naive search:
  DNA length N = 10,000
  Pattern length M = 5
  Operations = N x M = 50,000 per patient per pattern
  100 patients x 5 patterns = 25,000,000 operations

KMP:
  Operations = N + M = 10,005 per patient per pattern
  100 patients x 5 patterns = 5,002,500 operations

  5x faster on our data
  At genome scale (3 billion chars) = millions times faster
```

LPS Array (Longest Proper Prefix = Suffix)

The key to KMP. Built once from the pattern. Tells us: when mismatch happens, how far back in the PATTERN can we jump without re-checking DNA characters?

```
Pattern = "ABCAB"
LPS:      [0, 0, 0, 1, 2]

LPS[4] = 2 means:
  In "ABCAB", the prefix "AB" = suffix "AB"
  If mismatch at position 5, we already know
  first 2 chars match → restart from position 2
  not from position 0!

Building LPS (Abdul Bari method):
  i = 1 (moves forward always)
  j = 0 (tracks matching prefix length)

  pattern[i] == pattern[j] → LPS[i] = j+1, i++, j++
  mismatch, j > 0          → j = LPS[j-1]  (fall back)
  mismatch, j == 0         → LPS[i] = 0, i++
```

KMP Search (1-based indexing, Abdul Bari style)

```
text    starts at index 1 (dummy at 0)
pattern starts at index 1 (dummy at 0)
i moves on DNA text
j starts at 0, compare text[i] with pattern[j+1]

text[i] == pattern[j+1]  → i++, j++
```

```
mismatch, j > 0           → j = LPS[j]
mismatch, j == 0          → i++
j == pattern_length       → FOUND at position i - m
                              then j = LPS[j-1]  (find more)
```

Header file kmp.h

```
#ifndef KMP_H
#define KMP_H

vector buildLPS(string pattern);
vector KMPsearch(string text, string pattern);

#endif

#ifndef / #define / #endif = header guards
  Prevents same file being included twice
  If included twice without guards: function declared twice = error
  With guards: second include is skipped automatically
```

6. rabin_karp.cpp / rabin_karp.h

Key difference from KMP

	KMP	Rabin-Karp
Preprocessing	LPS array from pattern	Hash of patterns
Search method	Character comparison	Hash comparison (O(1))
Best for	1 pattern	Multiple patterns
Our data (1000 chars)	Won (less setup overhead)	Lost (more setup)
Genome scale (3B chars)	Would be slower	Would win (O(1) slides)

Rolling Hash

```
DNA has 4 bases: A=1, T=2, C=3, G=4

Hash of "GGT" (length 3):
  G × 4^2 + G × 4^1 + T × 4^0
  = 4×16 + 4×4 + 2×1
  = 64 + 16 + 2 = 82

When window slides from "GGT" to "GTA":
  OLD: remove first char G contribution
       82 - charVal('G') × 4^2 = 82 - 64 = 18
  SHIFT: multiply by base
         18 × 4 = 72
  NEW: add next char A
       72 + charVal('A') = 72 + 1 = 73

3 operations instead of recomputing from scratch!
This is O(1) per slide
```

Multi-pattern optimization — group by length

```
Our 5 mutations:
  "GGT"    length 3
  "CGGA"   length 4
  "TAGG"   length 4
  "ATCGG"  length 5
  "TTTCGA" length 6

Grouped:
  length 3 → {"GGT"}           → 1 pass
  length 4 → {"CGGA", "TAGG"} → 1 pass (checks BOTH simultaneously!)
  length 5 → {"ATCGG"}         → 1 pass
  length 6 → {"TTTCGA"}        → 1 pass
```

```
  Total: 4 passes instead of 5

At each window position for length 4:
  compute winHash
  check if winHash matches CGGA hash OR TAGG hash
  O(1) lookup checks both at once!
```

Spurious hits

A spurious hit is when the hash of a window matches the hash of the pattern, but the actual strings are different. We always verify character by character after a hash match to catch this.

7. main.cpp — All Changes Explained

Change 1: New includes

```
#include   // for freqMap (global frequencies)
#include           // for priority_queue (top-K mutations)
#include          // for timing both algorithms
#include "patient.h"      // Patient struct (moved from main.cpp)
#include "db.h"           // insertToMySQL function
#include "rabin_karp.h"   // RabinKarpMulti function
```

Change 2: Patient struct removed

The struct block that was in main.cpp was deleted. It now lives in patient.h and is accessed via #include. This way db.cpp and main.cpp both use the same definition without duplication.

Change 3: Timing added

```
auto kmpStart = chrono::high_resolution_clock::now();
// KMP loop runs here
auto kmpEnd   = chrono::high_resolution_clock::now();
double kmpTime = chrono::duration(kmpEnd - kmpStart).count();

high_resolution_clock::now()  → captures exact current time
kmpEnd - kmpStart             → difference = how long KMP took
duration       → convert to milliseconds (decimal)
.count()                      → extract plain number from duration object

Same done for Rabin-Karp with rkStart and rkEnd
```

Change 4: computeStats() function

```
void computeStats(Patient& p) {
    p.uniqueMutations = p.mutationMap.size();
    // size() = number of keys = number of different mutation types found

    p.mutationDensity = (double)p.totalMutations / p.dna.length() * 1000.0;
    // formula: (total / DNA length) x 1000
    // example: (21 / 1000) x 1000 = 21.0
    // meaning: 21 mutations per 1000 DNA characters

    p.riskScore = (p.totalMutations * 0.5)
                + (p.uniqueMutations * 1.0)
                + (p.mutationDensity * 0.3);
    // weights:
    // 0.5 → total count (repeating same mutation less dangerous)
    // 1.0 → unique types (diversity = more dangerous, highest weight)
```

```
    // 0.3 → density (concentration, lowest weight)
}
```

Change 5: Global frequency tracking

```
unordered_map freqMap;
// unordered_map chosen here (not map) because:
//   5 mutation names → order doesn't matter
//   O(1) lookup vs O(log n) for map
//   Only 5 keys so both are fast anyway

void computeFrequencies(vector& patients,
                        unordered_map& freqMap) {
    for (auto& p : patients) {
        for (auto& entry : p.mutationMap) {
            freqMap[entry.first] += entry.second.size();
            // entry.first  = "GGT"   (mutation name)
            // entry.second = [23,145] (positions)
            // .size()      = 2        (count for this patient)
            // freqMap["GGT"] accumulates across all patients
        }
    }
}

After all 100 patients:
  freqMap["GGT"]    = 1637 (total across all patients)
  freqMap["TAGG"]   = 448
  freqMap["CGGA"]   = 427
```

Change 6: Top-K with priority_queue

```
priority_queue> pq;
// max-heap: highest value always at top
// pair: first=frequency (used for sorting), second=name

for (auto& entry : freqMap) {
    pq.push({entry.second, entry.first});
    // {1637, "GGT"}, {448, "TAGG"} etc
    // int comes first so sorted by frequency
}

// pop top-K elements
while (!pq.empty() && rank <= k) {
    auto top = pq.top();   // highest frequency
    pq.pop();              // remove it
    cout << top.second << " → " << top.first << "\n";
    rank++;
```

```
}

Result:
  1. GGT    → 1637
  2. TAGG   → 448
  3. CGGA   → 427
  4. ATCGG  → 132
  5. TTTCGA → 36
```

Change 7: MySQL call at end of main

```
insertToMySQL(patients, freqMap);
// One line at bottom of main()
// Calls db.cpp which writes 3 CSV files
// Python then reads CSV and inserts into MySQL
```

8. db.cpp / db.h — Writing CSV Files

Originally this file was supposed to connect C++ directly to MySQL. That failed due to architecture issues (explained in Section 11). Now it writes 3 CSV files which Python then reads.

Three CSV files written

File	Columns	Rows
results.csv	patient_id, age, dna_length, total_mutations, unique_mutations, mutation_density, risk_score	100 (one per patient)
mutation_results.csv	patient_id, mutation_name, position, algorithm	Thousands (one per mutation occurrence)
frequencies.csv	mutation_name, total_count	5 (one per mutation type)

Writing patients CSV

```
ofstream pFile("results.csv");
// ofstream = output file stream (write to file)
// "results.csv" = filename to create

pFile << "patient_id,age,...\n";
// First line = header (column names)

for (auto& p : patients) {
    pFile << p.id << "," << p.age << "," << p.dna.length() << ...;
    // one line per patient
    // comma separated values
}
pFile.close();
// MUST close to flush buffer to disk
// Without close: last lines might not be written
```

Writing mutation results CSV — 3 nested loops

```
for (auto& p : patients) {           // loop 1: each patient
    for (auto& entry : p.mutationMap) {  // loop 2: each mutation type
        for (int pos : entry.second) {   // loop 3: each position
            mFile << p.id        << ","
                  << entry.first << ","  // mutation name "GGT"
                  << pos         << ","  // position 92
                  << "KMP"       << "\n";
        }
    }
}

Example output for P001:
  P001,GGT,92,KMP
```

```
P001,GGT,167,KMP
P001,GGT,246,KMP
P001,ATCGG,726,KMP
P001,TAGG,324,KMP
...
```

9. insert_db.py — Python MySQL Insertion

Reads the 3 CSV files created by C++ and inserts all data into MySQL. Python chosen because mysql-connector-python works regardless of 32/64 bit architecture issues.

Connecting to MySQL

```
import mysql.connector
import csv

conn = mysql.connector.connect(
    host="localhost",       # MySQL server on YOUR computer
    user="root",            # default MySQL admin username
    password="...",         # your MySQL password set during install
    database="genomic_tracker"  # which database to use
)
cursor = conn.cursor()
# conn = door to MySQL database
# cursor = pen to write SQL queries
```

Clearing old data — ORDER MATTERS

```
cursor.execute("DELETE FROM mutation_results")   # child first!
cursor.execute("DELETE FROM patients")             # parent second
cursor.execute("DELETE FROM mutation_frequencies")
conn.commit()

WHY this order?
  mutation_results has FOREIGN KEY → patients.patient_id

  If we DELETE FROM patients first:
    MySQL says "mutation_results still references these patients!"
    ERROR: foreign key constraint fails

  Always delete CHILD table first, then PARENT table

conn.commit():
  Think of it like pressing Save
  Without commit: changes exist in memory but not saved to disk
  With commit: permanently written to MySQL
```

Reading CSV and inserting patients

```
with open("results.csv", "r") as f:
    reader = csv.DictReader(f)
    # DictReader reads each row as a dictionary:
```

```
    # row = {"patient_id": "P001", "age": "58", ...}
    # note: everything is STRING from CSV

    for row in reader:
        cursor.execute("""
            INSERT INTO patients VALUES (%s,%s,%s,%s,%s,%s,%s)
        """, (
            row["patient_id"],          # string, stays string
            int(row["age"]),             # "58" → 58 (int)
            int(row["dna_length"]),
            int(row["total_mutations"]),
            int(row["unique_mutations"]),
            float(row["mutation_density"]),  # "21.0" → 21.0 (float)
            float(row["risk_score"])
        ))

%s = placeholder (safer than string formatting)
     Python fills in actual values
     Prevents SQL injection attacks
```

Reading mutation results CSV

```
with open("mutation_results.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        cursor.execute("""
            INSERT INTO mutation_results
            (patient_id, mutation_name, position, algorithm)
            VALUES (%s,%s,%s,%s)
        """, (
            row["patient_id"],
            row["mutation_name"],
            int(row["position"]),
            row["algorithm"]
        ))
conn.commit()

Note: we specify column names in INSERT because
id column is AUTO_INCREMENT — MySQL fills it automatically
We don't pass a value for id
```

Reading frequencies CSV

```
with open("frequencies.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        cursor.execute("""
```

```
            INSERT INTO mutation_frequencies VALUES (%s,%s)
        """, (
            row["mutation_name"],
            int(row["total_count"])
        ))
conn.commit()

cursor.close()  # release the pen
conn.close()    # close the door to MySQL
```

10. setup.sql — Database Design

Creates the database and 3 tables. Also contains all 5 analytics queries.

Database and table creation

```
CREATE DATABASE IF NOT EXISTS genomic_tracker;
USE genomic_tracker;
-- IF NOT EXISTS: don't crash if already exists, just skip

CREATE TABLE patients (
    patient_id       VARCHAR(10)  PRIMARY KEY,
    -- VARCHAR(10): string up to 10 chars ("P001" = 4 chars, fits)
    -- PRIMARY KEY: unique identifier, no two patients same ID

    age              INT,
    dna_length       INT,
    total_mutations  INT,
    unique_mutations INT,
    mutation_density DOUBLE,   -- decimal number (21.0)
    risk_score       DOUBLE    -- decimal number (20.8)
);
```

```
CREATE TABLE mutation_results (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    -- AUTO_INCREMENT: MySQL fills this automatically (1,2,3,4...)
    -- We never pass a value for id

    patient_id    VARCHAR(10),
    mutation_name VARCHAR(20),
    position      INT,
    algorithm     VARCHAR(20),

    FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
    -- Links to patients table
    -- Cannot insert mutation result for non-existent patient
    -- Ensures data integrity
);
```

```
CREATE TABLE mutation_frequencies (
    mutation_name VARCHAR(20) PRIMARY KEY,
    total_count   INT
);
-- One row per mutation type
-- mutation_name is PRIMARY KEY: each mutation appears once
```

Table relationships

```
patients (parent)
  patient_id PK
      |
      | (one patient → many mutation results)
      |
mutation_results (child)
  id PK (auto)
  patient_id FK → references patients.patient_id
  mutation_name
  position
  algorithm

mutation_frequencies (independent)
  mutation_name PK
  total_count
```

11. MySQL — Runtime vs Compile Time Issue

What we tried first

```
g++ main.cpp db.cpp -lmysql

This failed with:
  libmysql.dll: file not recognized: File format not recognized
```

Runtime vs Compile Time files

```
COMPILE TIME files (used by g++ during compilation):
  .h   files  → header declarations
  .lib files  → import libraries (tell linker where functions are)

RUNTIME files (used by Windows when .exe runs):
  .dll files  → dynamic link libraries (actual code, loaded at runtime)

When we used -lmysql:
  g++ looked for libmysql.lib (compile time)
  but found libmysql.dll (runtime)
  g++ said: "this is not a compile-time file, I cannot use it"

The .dll is loaded automatically by Windows when you run tracker.exe
You cannot link it with g++ at compile time
```

The architecture mismatch

```
We then tried: -lmysqlclient (using mysqlclient.lib)

Error:
  unhandled machine type 0x8664

0x8664 = x86_64 = 64-bit
Your g++ = MinGW 32-bit

MySQL 8.0 on Windows = compiled for 64-bit
MinGW g++ = 32-bit compiler

A 32-bit compiler CANNOT link against 64-bit libraries
It's like trying to plug EU socket into US outlet
Same electricity, incompatible physical shape
```

Why Python fixes this

```
pip install mysql-connector-python
```

```
mysql-connector-python is compiled by the Python team
It handles all architecture differences internally
Python 3.12 on your machine = 64-bit Python
mysql-connector = 64-bit compatible
No linker issues, no architecture conflicts

This is why polyglot architecture is used in real systems:
  Use each language for what it does best
  C++    → raw speed for algorithms
  Python → ecosystem of libraries that just work
```

The final solution

```
C++ db.cpp: writes 3 CSV files (no MySQL connection)
  results.csv
  mutation_results.csv
  frequencies.csv

Python insert_db.py: reads CSV → inserts MySQL
  No architecture issues
  Simple, readable code

Interview answer:
  "MinGW g++ on Windows is 32-bit but MySQL 8.0 is 64-bit.
   The .lib files are architecture-incompatible at link time.
   Python's mysql-connector handles this internally.
   This is polyglot architecture — industry standard practice."
```

12. SQL Analytics Queries Explained

Query 1: Top 10 highest risk patients

```
SELECT patient_id, age, risk_score
FROM patients
ORDER BY risk_score DESC
LIMIT 10;

SELECT    → which columns to show
FROM      → which table
ORDER BY  → sort results
DESC      → highest first (descending)
LIMIT 10  → only top 10 rows

Result shows: P068, P061 with risk 38.6 at top
```

Query 2: Average mutations by age group

```
SELECT
    CASE
        WHEN age < 30 THEN '20-29'
        WHEN age < 50 THEN '30-49'
        WHEN age < 70 THEN '50-69'
        ELSE '70-80'
    END AS age_group,
    COUNT(*) as total_patients,
    AVG(total_mutations) as avg_mutations,
    AVG(risk_score) as avg_risk
FROM patients
GROUP BY age_group
ORDER BY age_group;

CASE/WHEN/THEN/ELSE/END:
  Like if-else in SQL
  age=25 → '20-29'
  age=45 → '30-49'

AS age_group:
  gives the CASE result a column name

COUNT(*): count patients in each group
AVG():    average value across group

GROUP BY age_group:
  splits patients into 4 buckets
  COUNT and AVG computed per bucket
```

Query 3: Most common mutations

```
SELECT mutation_name, total_count
FROM mutation_frequencies
ORDER BY total_count DESC;

Simple! Just reads mutation_frequencies table
sorted by count highest first

Result:
  GGT    → 1637
  TAGG   → 448
  CGGA   → 427
  ATCGG  → 132
  TTTCGA → 36
```

Query 4: Window function — rank patients by risk

```
SELECT
    patient_id,
    age,
    risk_score,
    RANK() OVER (ORDER BY risk_score DESC) as risk_rank,
    AVG(risk_score) OVER () as overall_avg_risk
FROM patients
LIMIT 10;

WINDOW FUNCTIONS — what makes this advanced SQL:

RANK() OVER (ORDER BY risk_score DESC):
  Assigns a rank to each patient
  Highest risk_score gets rank 1
  OVER = window function keyword
  Handles TIES: two patients with 38.6 both get rank 1
  Next patient gets rank 3 (skips 2)

AVG(risk_score) OVER ():
  Computes average risk across ALL patients
  Shows same value (25.45) for every row
  Without OVER, AVG would collapse all rows into one
  With OVER (), it calculates across the whole table
  but shows result on every row

Why window functions impress recruiters:
  Regular AVG, COUNT, SUM collapse rows
  Window functions keep all rows
  AND add computed column
  This is senior-level SQL
```

Query 5: Age group density comparison

```
SELECT
    CASE ... END AS age_group,
    AVG(mutation_density) as avg_density,
    MAX(risk_score) as max_risk,
    MIN(risk_score) as min_risk
FROM patients
GROUP BY age_group
ORDER BY avg_density DESC;

MAX() and MIN() per age group:
  Shows range of risk scores within each group
  How spread out the risk is

ORDER BY avg_density DESC:
  Age group with highest mutation density shown first

Result shows which age group is most densely mutated
```

13. Interview Q&A;

Most likely questions a recruiter will ask about this project.

Q: Tell me about your project

A: I built a genomic mutation tracker that searches patient DNA sequences for disease-causing patterns using two string algorithms. I implemented KMP and Rabin-Karp in C++, benchmarked them, computed patient risk scores, and stored results in MySQL for analytics using window functions.

Q: Why two algorithms?

A: To demonstrate tradeoffs. KMP is optimal for single pattern search using LPS preprocessing — O(N+M). Rabin-Karp uses rolling hash which is better for multiple patterns simultaneously. On our 1000-char DNA, KMP won due to lower setup overhead. At genome scale (3 billion chars, 50+ patterns), Rabin-Karp's O(1) rolling hash advantage would dominate.

Q: What is LPS?

A: Longest Proper Prefix which is also a Suffix. For each position in the pattern, it stores the length of the longest prefix that also appears as a suffix. When a mismatch occurs in KMP, instead of restarting from scratch, we jump back in the PATTERN only as far as LPS tells us — without ever moving backwards in the text. This makes KMP O(N+M) instead of O(N*M).

Q: What is rolling hash?

A: A technique to compute a new window's hash from the previous window's hash in O(1) instead of O(M). We subtract the contribution of the first character, shift by multiplying by the base (4 for DNA), and add the new character. No need to recompute from scratch. A hash match triggers character-by-character verification to catch spurious hits.

Q: Why Python for MySQL?

A: MinGW g++ on Windows is 32-bit but MySQL 8.0 is compiled for 64-bit. The .lib files are architecture-incompatible at link time — a 32-bit linker cannot use 64-bit libraries. Python's mysql-connector handles this internally. This polyglot approach — C++ for speed, Python for database — is the industry standard.

Q: Is your data realistic?

A: It's a simulation. Mutation injection correlates with age to model the biological reality that older patients accumulate more DNA copying errors. The algorithms are identical regardless of data source. In production, we'd use NCBI genomic datasets. I documented this limitation in the README — acknowledging limitations

shows engineering maturity.

Q: What is the risk score?

A: A weighted combination of three factors: total mutation count (weight 0.5), unique mutation types (weight 1.0 — highest because diversity is more dangerous than repetition), and mutation density (weight 0.3). It's a simplified model — production would use clinically validated weights from published research.

Q: What SQL concepts did you use?

A: CASE statements for age bucketing, GROUP BY for aggregation, ORDER BY for ranking, RANK() window function for patient ranking with tied-score handling, AVG() OVER() for running averages across all rows, FOREIGN KEY constraints for referential integrity, and AUTO_INCREMENT for automatic ID generation.

Q: What data structures did you use?

A: vector for dynamic patient list, map for mutation→positions mapping (sorted by name), unordered_map for global frequency counts (O(1) lookup), priority_queue for top-K mutation extraction (max-heap), and struct to group patient data. Each was chosen for its specific access pattern.

Q: Why map not unordered_map for mutationMap?

A: Only 5 mutations so performance difference is negligible. Map maintains alphabetical key order, making output consistently sorted — ATCGG before GGT before TAGG every time. Easier to read and verify. For 10,000+ mutations, unordered_map would be the choice.

14. Git Commands Reminder

Study these before any interview. These are the ones every developer uses daily.

Command	What it does	When to use
`git init`	Start tracking a folder with git	Once per project, at the beginning
`git add .`	Stage ALL changed files	Before every commit
`git add filename`	Stage ONE specific file	When you only want to commit one file
`git commit -m "msg"`	Save a snapshot with a description	After git add, before git push
`git push`	Send commits to GitHub	After git commit
`git pull`	Get latest changes from GitHub	Before starting work each day
`git clone URL`	Download a GitHub repo to your computer	When starting to work on an existing project
`git status`	See which files changed or are staged	Any time you're unsure
`git log`	See history of all commits	To see what was done before
`git rm --cached file`	Stop tracking a file (keep on disk)	When a file was accidentally committed
`git branch`	List or create branches	When working on a new feature
`git checkout -b name`	Create and switch to new branch	Before starting a new feature
`git merge branch`	Combine a branch into current branch	After feature is complete

The most important interview question about git

```
Q: What is the difference between Git and GitHub?

A: Git is a tool installed on your computer that tracks changes
   to your code. GitHub is a website that stores your code online.

   Git can work without GitHub (local tracking only).
   GitHub needs Git to push/pull code.

   Git  = the version control system (tool)
   GitHub = the hosting platform (website)

   Other hosting platforms: GitLab, Bitbucket (same concept)
```

The 3-step workflow you always use

```
git add .          → Stage: put files in the box
git commit -m "..." → Commit: seal the box with a label
git push           → Push: send the box to GitHub

Never skip a step. Always in this order.
```

END OF GUIDE

You built a complete end-to-end system covering string algorithms, data structures, file I/O, database design, SQL analytics, and version control — all in one project. That's impressive work.